SC1001: Introduction to Computational Thinking

Week 10: Recursion on Binary Trees

Jordan Boyd-Graber

Nanyang Technological University

Slides

Slides and updates will be posted at

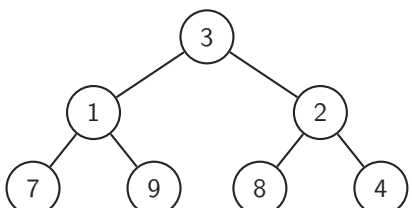
`boydgraber.org/teaching/SC_1001/`

Overview

1. Start from recursive algorithms for complete binary trees
2. Extend the representation to missing children
3. Add the empty-tree base case
4. Reuse the same recursive decomposition

Complete Binary Trees

A complete binary tree representation uses a list with three parts.

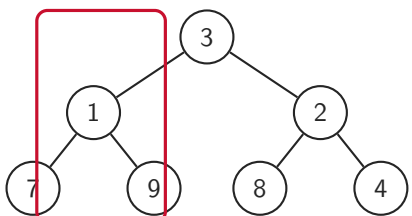



```
tree = [[[7], 1, [9]], 3, [[8], 2, [4]]]
```

Each internal node uses `[left, root, right]`.

Complete Binary Trees

A complete binary tree representation uses a list with three parts.

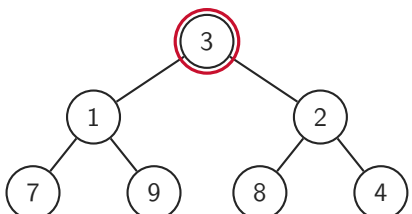



```
tree = [ [[7], 1, [9]] , 3, [[8], 2, [4]]]
```

The first element is the left subtree rooted at `1`.

Complete Binary Trees

A complete binary tree representation uses a list with three parts.

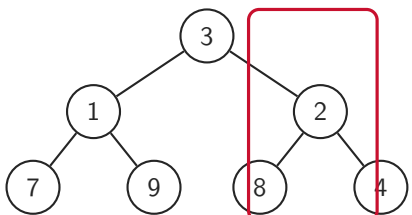



```
tree = [[[7], 1, [9]], 3, [[8], 2, [4]]]
```

The middle element is the root value 3.

Complete Binary Trees

A complete binary tree representation uses a list with three parts.




```
tree = [[[7], 1, [9]], 3, [[8], 2, [4]] ]
```

The last element is the right subtree rooted at 2.

Type Hints

```
from typing import TypeAlias

Tree: TypeAlias = list[] | list[int] | list["Tree", int, "Tree"]

# Tree values must have one of these shapes:
# []                  empty tree
# [root]              leaf node
# [left, root, right] internal node
```

- Function names use `snake_case`.
- Internal nodes always place an `int` between two trees.
- The docstring says what the function returns.

Reference: count_nodes

```
def count_nodes(tree: Tree) -> int:
    """Returns the number of nodes in tree."""
    if len(tree) == 1:
        return 1
    left, root, right = tree
    return count_nodes(left) + count_nodes(right) + 1
```

The function receives one tree and returns an integer.

Reference: count_nodes

```
def count_nodes(tree: Tree) -> int:
    """Returns the number of nodes in tree."""
    if len(tree) == 1:
        return 1
    left, root, right = tree
    return count_nodes(left) + count_nodes(right) + 1
```

The docstring describes the result.

Reference: count_nodes

```
def count_nodes(tree: Tree) -> int:
    """Returns the number of nodes in tree."""
    if len(tree) == 1:
        return 1
    left, root, right = tree
    return count_nodes(left) + count_nodes(right) + 1
```

A complete-tree leaf has length 1.

Reference: count_nodes

```
def count_nodes(tree: Tree) -> int:
    """Returns the number of nodes in tree."""
    if len(tree) == 1:
        return 1
    left, root, right = tree
    return count_nodes(left) + count_nodes(right) + 1
```

A leaf contains exactly one node.

Reference: count_nodes

```
def count_nodes(tree: Tree) -> int:
    """Returns the number of nodes in tree."""
    if len(tree) == 1:
        return 1
    left, root, right = tree
    return count_nodes(left) + count_nodes(right) + 1
```

Unpack the internal node into its three parts.

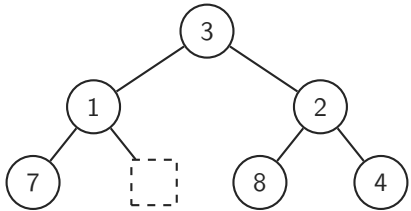
Reference: count_nodes

```
def count_nodes(tree: Tree) -> int:
    """Returns the number of nodes in tree."""
    if len(tree) == 1:
        return 1
    left, root, right = tree
    return count_nodes(left) + count_nodes(right) + 1
```

Count the left subtree, right subtree, and root.

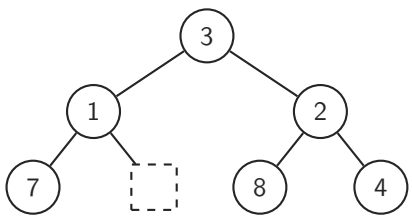
Discussion 1: Binary Tree Representation

How could we extend the list representation to binary trees where a parent node may have one child or two children?



Binary Tree Representation

Use the same three-part list, and use an empty list when a child is missing.

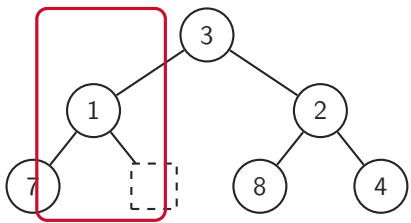



```
tree = [[[7], 1, []], 3, [[8], 2, [4]]]
```

The overall shape still uses `[left, root, right]`.

Binary Tree Representation

Use the same three-part list, and use an empty list when a child is missing.

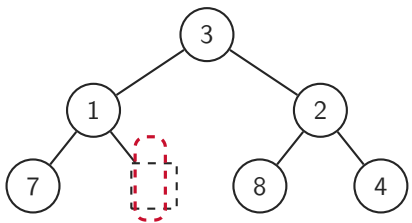



```
tree = [ [[7], 1, []] , 3, [[8], 2, [4]]]
```

The left subtree is rooted at `1`.

Binary Tree Representation

Use the same three-part list, and use an empty list when a child is missing.

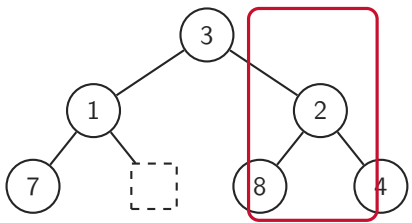



```
tree = [[[7], 1, [] ], 3, [[8], 2, [4]]]
```

The empty list marks the missing right child of `1`.

Binary Tree Representation

Use the same three-part list, and use an empty list when a child is missing.

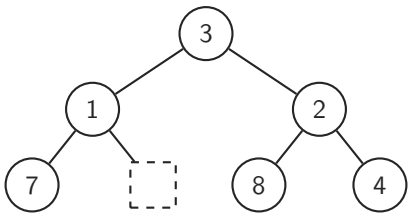



```
tree = [[[7], 1, []], 3, [[8], 2, [4]] ]
```

The right subtree is still a normal subtree.

Discussion 2: Count Nodes

How do we extend `count_nodes(tree)` to allow empty children?



count_nodes: Add Empty Tree

Reference

```
def count_nodes(tree: Tree) -> int:
    """Returns the number of nodes in tree."""
    if len(tree) == 1:
        return 1
    left, root, right = tree
    return count_nodes(left) + count_nodes(right) + 1
```

Extended

Start from the complete-tree reference code. The extended code appears on the next advance.

count_nodes: Add Empty Tree

Reference

```
def count_nodes(tree: Tree) -> int:
    """Returns the number of nodes in tree."""
    if len(tree) == 1:
        return 1
    left, root, right = tree
    return count_nodes(left) + count_nodes(right) + 1
```

Extended

```
def count_nodes(tree: Tree) -> int:
    """Returns the number of nodes in tree."""
```

Keep the same function signature and docstring.

count_nodes: Add Empty Tree

Reference

```
def count_nodes(tree: Tree) -> int:
    """Returns the number of nodes in tree."""
    if len(tree) == 1:
        return 1
    left, root, right = tree
    return count_nodes(left) + count_nodes(right) + 1
```

Extended

```
def count_nodes(tree: Tree) -> int:
    """Returns the number of nodes in tree."""
    if len(tree) == 1:
        return 1
```

The leaf base case stays the same.

count_nodes: Add Empty Tree

Reference

```
def count_nodes(tree: Tree) -> int:
    """Returns the number of nodes in tree."""
    if len(tree) == 1:
        return 1
    left, root, right = tree
    return count_nodes(left) + count_nodes(right) + 1
```

Extended

```
def count_nodes(tree: Tree) -> int:
    """Returns the number of nodes in tree."""
    if len(tree) == 1:
        return 1
    if len(tree) == 0:
        return 0
```

Add the empty-tree base case. `[]` contributes zero nodes.

count_nodes: Add Empty Tree

Reference

```
def count_nodes(tree: Tree) -> int:
    """Returns the number of nodes in tree."""
    if len(tree) == 1:
        return 1
    left, root, right = tree
    return count_nodes(left) + count_nodes(right) + 1
```

Extended

```
def count_nodes(tree: Tree) -> int:
    """Returns the number of nodes in tree."""
    if len(tree) == 1:
        return 1
    if len(tree) == 0:
        return 0
    left, root, right = tree
    return count_nodes(left) + count_nodes(right) + 1
```

The recursive computation still counts left subtree, right subtree, and root.

Discussion 3: Sum Nodes

How do we extend `sum_nodes(tree)` to allow empty children?

Reference: sum_nodes

```
def sum_nodes(tree: Tree) -> int:
    """Returns the sum of all node values in tree."""
    if len(tree) == 1:
        return tree[0]
    left, root, right = tree
    return sum_nodes(left) + root + sum_nodes(right)
```

The function receives one tree and returns an integer.

Reference: sum_nodes

```
def sum_nodes(tree: Tree) -> int:
    """Returns the sum of all node values in tree."""
    if len(tree) == 1:
        return tree[0]
    left, root, right = tree
    return sum_nodes(left) + root + sum_nodes(right)
```

The docstring gives the goal.

Reference: sum_nodes

```
def sum_nodes(tree: Tree) -> int:
    """Returns the sum of all node values in tree."""
    if len(tree) == 1:
        return tree[0]
    left, root, right = tree
    return sum_nodes(left) + root + sum_nodes(right)
```

A leaf is the base case.

Reference: sum_nodes

```
def sum_nodes(tree: Tree) -> int:
    """Returns the sum of all node values in tree."""
    if len(tree) == 1:
        return tree[0]
    left, root, right = tree
    return sum_nodes(left) + root + sum_nodes(right)
```

For a leaf, the only value is stored at index 0.

Reference: sum_nodes

```
def sum_nodes(tree: Tree) -> int:
    """Returns the sum of all node values in tree."""
    if len(tree) == 1:
        return tree[0]
    left, root, right = tree
    return sum_nodes(left) + root + sum_nodes(right)
```

Unpack the internal node into left subtree, root, and right subtree.

Reference: sum_nodes

```
def sum_nodes(tree: Tree) -> int:
    """Returns the sum of all node values in tree."""
    if len(tree) == 1:
        return tree[0]
    left, root, right = tree
    return sum_nodes(left) + root + sum_nodes(right)
```

Add the left sum, root value, and right sum.

sum_nodes: Add Empty Tree

Reference

```
def sum_nodes(tree: Tree) -> int:
    """Returns the sum of all node values in tree."""
    if len(tree) == 1:
        return tree[0]
    left, root, right = tree
    return sum_nodes(left) + root + sum_nodes(right)
```

Extended

Start from the complete-tree reference code. The extended code appears on the next advance.

sum_nodes: Add Empty Tree

Reference

```
def sum_nodes(tree: Tree) -> int:
    """Returns the sum of all node values in tree."""
    if len(tree) == 1:
        return tree[0]
    left, root, right = tree
    return sum_nodes(left) + root + sum_nodes(right)
```

Extended

```
def sum_nodes(tree: Tree) -> int:
    """Returns the sum of all node values in tree."""
```

Keep the same function signature and docstring.

sum_nodes: Add Empty Tree

Reference

```
def sum_nodes(tree: Tree) -> int:
    """Returns the sum of all node values in tree."""
    if len(tree) == 1:
        return tree[0]
    left, root, right = tree
    return sum_nodes(left) + root + sum_nodes(right)
```

Extended

```
def sum_nodes(tree: Tree) -> int:
    """Returns the sum of all node values in tree."""
    if len(tree) == 1:
        return tree[0]
```

The leaf case returns its only value.

sum_nodes: Add Empty Tree

Reference

```
def sum_nodes(tree: Tree) -> int:
    """Returns the sum of all node values in tree."""
    if len(tree) == 1:
        return tree[0]
    left, root, right = tree
    return sum_nodes(left) + root + sum_nodes(right)
```

Extended

```
def sum_nodes(tree: Tree) -> int:
    """Returns the sum of all node values in tree."""
    if len(tree) == 1:
        return tree[0]
    if len(tree) == 0:
        return 0
```

An empty child contributes nothing to the sum.

sum_nodes: Add Empty Tree

Reference

```
def sum_nodes(tree: Tree) -> int:
    """Returns the sum of all node values in tree."""
    if len(tree) == 1:
        return tree[0]
    left, root, right = tree
    return sum_nodes(left) + root + sum_nodes(right)
```

Extended

```
def sum_nodes(tree: Tree) -> int:
    """Returns the sum of all node values in tree."""
    if len(tree) == 1:
        return tree[0]
    if len(tree) == 0:
        return 0
    left, root, right = tree
    return sum_nodes(left) + root + sum_nodes(right)
```

The recursive case still adds left subtree, root value, and right subtree.

Discussion 4: Maximum Node

How do we extend `max_node(tree)` to allow empty children?

What should an empty tree return so it never wins the maximum comparison?

Reference: max_node

```
def max_node(tree: Tree) -> int | float:
    """Returns the largest node value in tree."""
    if len(tree) == 1:
        return tree[0]
    left, root, right = tree
    return max(max_node(left), root, max_node(right))
```

The function receives a tree and returns the largest value.

Reference: `max_node`

```
def max_node(tree: Tree) -> int | float:
    """Returns the largest node value in tree."""
    if len(tree) == 1:
        return tree[0]
    left, root, right = tree
    return max(max_node(left), root, max_node(right))
```

The docstring states the comparison we need.

Reference: max_node

```
def max_node(tree: Tree) -> int | float:
    """Returns the largest node value in tree."""
    if len(tree) == 1:
        return tree[0]
    left, root, right = tree
    return max(max_node(left), root, max_node(right))
```

A leaf is the base case.

Reference: max_node

```
def max_node(tree: Tree) -> int | float:
    """Returns the largest node value in tree."""
    if len(tree) == 1:
        return tree[0]
    left, root, right = tree
    return max(max_node(left), root, max_node(right))
```

For a leaf, return its single value.

Reference: max_node

```
def max_node(tree: Tree) -> int | float:
    """Returns the largest node value in tree."""
    if len(tree) == 1:
        return tree[0]
    left, root, right = tree
    return max(max_node(left), root, max_node(right))
```

Unpack the internal node.

Reference: max_node

```
def max_node(tree: Tree) -> int | float:
    """Returns the largest node value in tree."""
    if len(tree) == 1:
        return tree[0]
    left, root, right = tree
    return max(max_node(left), root, max_node(right))
```

The maximum is the best among left subtree, root, and right subtree.

max_node: Add Empty Tree

Reference

```
def max_node(tree: Tree) -> int | float:
    """Returns the largest node value in tree."""
    if len(tree) == 1:
        return tree[0]
    left, root, right = tree
    return max(max_node(left), root, max_node(right))
```

Extended

Start from the complete-tree reference code. The extended code appears on the next advance.

max_node: Add Empty Tree

Reference

```
def max_node(tree: Tree) -> int | float:
    """Returns the largest node value in tree."""
    if len(tree) == 1:
        return tree[0]
    left, root, right = tree
    return max(max_node(left), root, max_node(right))
```

Extended

```
from math import inf

def max_node(tree: Tree) -> int | float:
```

Define a sentinel that never beats a real value in a maximum comparison.

max_node: Add Empty Tree

Reference

```
def max_node(tree: Tree) -> int | float:
    """Returns the largest node value in tree."""
    if len(tree) == 1:
        return tree[0]
    left, root, right = tree
    return max(max_node(left), root, max_node(right))
```

Extended

```
from math import inf

def max_node(tree: Tree) -> int | float:
    """Returns the largest node value in tree."""
    if len(tree) == 1:
        return tree[0]
```

The leaf base case stays the same.

max_node: Add Empty Tree

Reference

```
def max_node(tree: Tree) -> int | float:
    """Returns the largest node value in tree."""
    if len(tree) == 1:
        return tree[0]
    left, root, right = tree
    return max(max_node(left), root, max_node(right))
```

Extended

```
from math import inf

def max_node(tree: Tree) -> int | float:
    """Returns the largest node value in tree."""
    if len(tree) == 1:
        return tree[0]
    if len(tree) == 0:
        return -inf
```

Return negative infinity for the empty tree.

max_node: Add Empty Tree

Reference

```
def max_node(tree: Tree) -> int | float:
    """Returns the largest node value in tree."""
    if len(tree) == 1:
        return tree[0]
    left, root, right = tree
    return max(max_node(left), root, max_node(right))
```

Extended

```
from math import inf

def max_node(tree: Tree) -> int | float:
    """Returns the largest node value in tree."""
    if len(tree) == 1:
        return tree[0]
    if len(tree) == 0:
        return -inf
    left, root, right = tree
    return max(max_node(left), root, max_node(right))
```

The recursive case can use Python's `max` directly.

Why `-inf`?

Python integers can grow beyond the fixed-width limits used in many other languages.

```
from math import inf

-inf
inf
```

- In C or Java, a smallest integer sentinel can be reasonable
  - A fixed width sequence of bits
  - Can result in overflow
- Python does not have one smallest possible integer: uses as many bits as needed
- IEEE floating point defines infinity values for comparisons.
- `-inf` loses every greater-than comparison with real node values.

Discussion 5: Minimum Node

How do we extend `min_node(tree)` to allow empty children?

What should an empty tree return so it never wins the minimum comparison?

Reference: `min_node`

```
def min_node(tree: Tree) -> int | float:
    """Returns the smallest node value in tree."""
    if len(tree) == 1:
        return tree[0]
    left, root, right = tree
    return min(min_node(left), root, min_node(right))
```

The function receives a tree and returns the smallest value.

Reference: `min_node`

```
def min_node(tree: Tree) -> int | float:
    """Returns the smallest node value in tree."""
    if len(tree) == 1:
        return tree[0]
    left, root, right = tree
    return min(min_node(left), root, min_node(right))
```

The docstring states the comparison we need.

Reference: `min_node`

```
def min_node(tree: Tree) -> int | float:
    """Returns the smallest node value in tree."""
    if len(tree) == 1:
        return tree[0]
    left, root, right = tree
    return min(min_node(left), root, min_node(right))
```

A leaf is the base case.

Reference: `min_node`

```
def min_node(tree: Tree) -> int | float:
    """Returns the smallest node value in tree."""
    if len(tree) == 1:
        return tree[0]
    left, root, right = tree
    return min(min_node(left), root, min_node(right))
```

For a leaf, return its single value.

Reference: `min_node`

```
def min_node(tree: Tree) -> int | float:
    """Returns the smallest node value in tree."""
    if len(tree) == 1:
        return tree[0]
    left, root, right = tree
    return min(min_node(left), root, min_node(right))
```

Unpack the internal node.

Reference: `min_node`

```
def min_node(tree: Tree) -> int | float:
    """Returns the smallest node value in tree."""
    if len(tree) == 1:
        return tree[0]
    left, root, right = tree
    return min(min_node(left), root, min_node(right))
```

The minimum is the best among left subtree, root, and right subtree.

`min_node`: Add Empty Tree

Reference

```
def min_node(tree: Tree) -> int | float:
    """Returns the smallest node value in tree."""
    if len(tree) == 1:
        return tree[0]
    left, root, right = tree
    return min(min_node(left), root, min_node(right))
```

Extended

Start from the complete-tree reference code. The extended code appears on the next advance.

min_node: Add Empty Tree

Reference

```
def min_node(tree: Tree) -> int | float:
    """Returns the smallest node value in tree."""
    if len(tree) == 1:
        return tree[0]
    left, root, right = tree
    return min(min_node(left), root, min_node(right))
```

Extended

```
from math import inf

def min_node(tree: Tree) -> int | float:
```

Define a sentinel that never beats a real value in a minimum comparison.

`min_node`: Add Empty Tree

Reference

```
def min_node(tree: Tree) -> int | float:
    """Returns the smallest node value in tree."""
    if len(tree) == 1:
        return tree[0]
    left, root, right = tree
    return min(min_node(left), root, min_node(right))
```

Extended

```
from math import inf

def min_node(tree: Tree) -> int | float:
    """Returns the smallest node value in tree."""
    if len(tree) == 1:
        return tree[0]
```

The leaf base case stays the same.

`min_node`: Add Empty Tree

Reference

```
def min_node(tree: Tree) -> int | float:
    """Returns the smallest node value in tree."""
    if len(tree) == 1:
        return tree[0]
    left, root, right = tree
    return min(min_node(left), root, min_node(right))
```

Extended

```
from math import inf

def min_node(tree: Tree) -> int | float:
    """Returns the smallest node value in tree."""
    if len(tree) == 1:
        return tree[0]
    if len(tree) == 0:
        return inf
```

Return positive infinity for the empty tree.

min_node: Add Empty Tree

Reference

```
def min_node(tree: Tree) -> int | float:
    """Returns the smallest node value in tree."""
    if len(tree) == 1:
        return tree[0]
    left, root, right = tree
    return min(min_node(left), root, min_node(right))
```

Extended

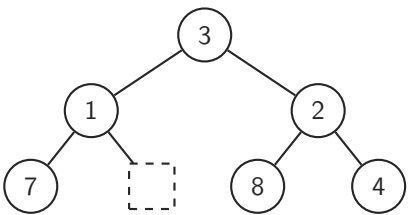
```
from math import inf

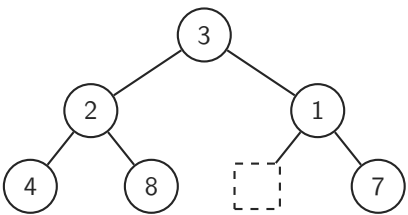
def min_node(tree: Tree) -> int | float:
    """Returns the smallest node value in tree."""
    if len(tree) == 1:
        return tree[0]
    if len(tree) == 0:
        return inf
    left, root, right = tree
    return min(min_node(left), root, min_node(right))
```

The recursive case can use Python's `min` directly.

Discussion 6: Mirror

How do we extend `mirror_tree(tree)` to allow empty children?





Reference: `mirror_tree`

```
def mirror_tree(tree: Tree) -> Tree:
    """Returns the mirror image of tree."""
    if len(tree) == 1:
        return tree
    left, root, right = tree
    return [mirror_tree(right), root, mirror_tree(left)]
```

The function receives a tree and returns a tree.

Reference: `mirror_tree`

```
def mirror_tree(tree: Tree) -> Tree:
    """Returns the mirror image of tree."""
    if len(tree) == 1:
        return tree
    left, root, right = tree
    return [mirror_tree(right), root, mirror_tree(left)]
```

The docstring names the transformation.

Reference: mirror_tree

```
def mirror_tree(tree: Tree) -> Tree:
    """Returns the mirror image of tree."""
    if len(tree) == 1:
        return tree
    left, root, right = tree
    return [mirror_tree(right), root, mirror_tree(left)]
```

A leaf is already its own mirror.

Reference: mirror_tree

```
def mirror_tree(tree: Tree) -> Tree:
    """Returns the mirror image of tree."""
    if len(tree) == 1:
        return tree
    left, root, right = tree
    return [mirror_tree(right), root, mirror_tree(left)]
```

Return the leaf unchanged.

Reference: `mirror_tree`

```
def mirror_tree(tree: Tree) -> Tree:
    """Returns the mirror image of tree."""
    if len(tree) == 1:
        return tree
    left, root, right = tree
    return [mirror_tree(right), root, mirror_tree(left)]
```

Unpack the three parts of the internal node.

Reference: `mirror_tree`

```
def mirror_tree(tree: Tree) -> Tree:
    """Returns the mirror image of tree."""
    if len(tree) == 1:
        return tree
    left, root, right = tree
    return [mirror_tree(right), root, mirror_tree(left)]
```

Mirror both subtrees, then swap left and right.

mirror_tree: Add Empty Tree

Reference

```
def mirror_tree(tree: Tree) -> Tree:
    """Returns the mirror image of tree."""
    if len(tree) == 1:
        return tree
    left, root, right = tree
    return [mirror_tree(right), root, mirror_tree(left)]
```

Extended

Start from the complete-tree reference code. The extended code appears on the next advance.

mirror_tree: Add Empty Tree

Reference

```
def mirror_tree(tree: Tree) -> Tree:
    """Returns the mirror image of tree."""
    if len(tree) == 1:
        return tree
    left, root, right = tree
    return [mirror_tree(right), root, mirror_tree(left)]
```

Extended

```
def mirror_tree(tree: Tree) -> Tree:
    """Returns the mirror image of tree."""
```

Keep the same function signature and docstring.

mirror_tree: Add Empty Tree

Reference

```
def mirror_tree(tree: Tree) -> Tree:
    """Returns the mirror image of tree."""
    if len(tree) == 1:
        return tree
    left, root, right = tree
    return [mirror_tree(right), root, mirror_tree(left)]
```

Extended

```
def mirror_tree(tree: Tree) -> Tree:
    """Returns the mirror image of tree."""
    if len(tree) == 1:
        return tree
```

A leaf still mirrors to itself.

mirror_tree: Add Empty Tree

Reference

```
def mirror_tree(tree: Tree) -> Tree:
    """Returns the mirror image of tree."""
    if len(tree) == 1:
        return tree
    left, root, right = tree
    return [mirror_tree(right), root, mirror_tree(left)]
```

Extended

```
def mirror_tree(tree: Tree) -> Tree:
    """Returns the mirror image of tree."""
    if len(tree) == 1:
        return tree
    if len(tree) == 0:
        return tree
```

An empty child also mirrors to itself.

mirror_tree: Add Empty Tree

Reference

```
def mirror_tree(tree: Tree) -> Tree:
    """Returns the mirror image of tree."""
    if len(tree) == 1:
        return tree
    left, root, right = tree
    return [mirror_tree(right), root, mirror_tree(left)]
```

Extended

```
def mirror_tree(tree: Tree) -> Tree:
    """Returns the mirror image of tree."""
    if len(tree) == 1:
        return tree
    if len(tree) == 0:
        return tree
    left, root, right = tree
    return [mirror_tree(right), root, mirror_tree(left)]
```

Mirror both subtrees, then swap them.

Exercise 1: Maximum Depth

Write `max_depth(tree)`.

- Empty tree depth is `0`.
- A leaf has depth `1`.
- An internal node is one level deeper than its deeper child.

max_depth

```
def max_depth(tree: Tree) -> int:
    """Returns the maximum depth of tree."""
```

Start with the function contract.

max_depth

```
def max_depth(tree: Tree) -> int:
    """Returns the maximum depth of tree."""
    if len(tree) == 0:
        return 0
```

The empty tree has depth zero.

max_depth

```
def max_depth(tree: Tree) -> int:
    """Returns the maximum depth of tree."""
    if len(tree) == 0:
        return 0
    if len(tree) == 1:
        return 1
```

A leaf has depth one.

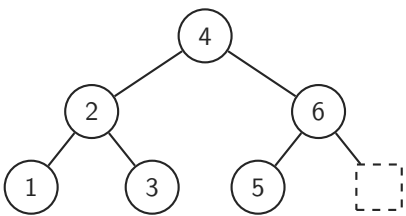
max_depth

```
def max_depth(tree: Tree) -> int:
    """Returns the maximum depth of tree."""
    if len(tree) == 0:
        return 0
    if len(tree) == 1:
        return 1
    left, root, right = tree
    return max(max_depth(left), max_depth(right)) + 1
```

An internal node is one level deeper than its deeper child.

Exercise 2: Binary Search Tree

A binary search tree stores smaller values on the left and larger values on the right.




```
bst = [[[1], 2, [3]], 4, [[5], 6, []]]
```

Write `insert_bst(tree, value)`. If `value` already appears, return the tree unchanged.

insert_bst

```
def insert_bst(tree: Tree, value: int) -> Tree:
    """Returns tree with value inserted as a BST node."""
```

Start with the function contract.

insert_bst

```
def insert_bst(tree: Tree, value: int) -> Tree:
    """Returns tree with value inserted as a BST node."""
    if len(tree) == 0:
        return [value]
```

Inserting into an empty tree creates a leaf.

insert_bst

```
def insert_bst(tree: Tree, value: int) -> Tree:
    """Returns tree with value inserted as a BST node."""
    if len(tree) == 0:
        return [value]
    if len(tree) == 1:
        root = tree[0]
        if value == root:
            return tree
        if value < root:
            return [[value], root, []]
        return [[], root, [value]]
```

For a leaf, duplicate values return the tree unchanged; smaller values go left and larger values go right.

insert_bst

```
def insert_bst(tree: Tree, value: int) -> Tree:
    """Returns tree with value inserted as a BST node."""
    if len(tree) == 0:
        return [value]
    if len(tree) == 1:
        root = tree[0]
        if value == root:
            return tree
        if value < root:
            return [[value], root, []]
        return [[], root, [value]]
    left, root, right = tree
    if value == root:
        return tree
```

For an internal node, unpack the three parts and stop on duplicates.

insert_bst

```
def insert_bst(tree: Tree, value: int) -> Tree:
    """Returns tree with value inserted as a BST node."""
    if len(tree) == 0:
        return [value]
    if len(tree) == 1:
        root = tree[0]
        if value == root:
            return tree
        if value < root:
            return [[value], root, []]
        return [[], root, [value]]
    left, root, right = tree
    if value == root:
        return tree
    if value < root:
        return [insert_bst(left, value), root, right]
    return [left, root, insert_bst(right, value)]
```

Recurse into the side where the new value belongs.

`insert_bst`: Invocation Trace



Start from the empty tree.

```
tree = []
```

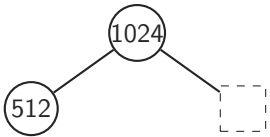
`insert_bst`: Invocation Trace




```
tree = insert_bst(tree, 1024)
tree == [1024]
```

The first value becomes the root leaf.

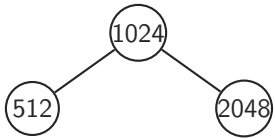
`insert_bst`: Invocation Trace



Since 512 < 1024, insert into the left child.

```
tree = insert_bst(tree, 512)
tree == [[512], 1024, []]
```

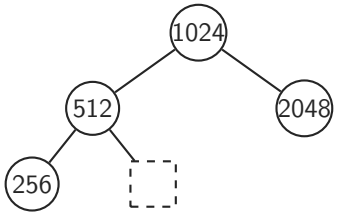
insert_bst: Invocation Trace




```
tree = insert_bst(tree, 2048)
tree == [[512], 1024, [2048]]
```

Since 2048 > 1024, insert into the right child.

insert_bst: Invocation Trace




```
tree = insert_bst(tree, 256)
tree == [[[256], 512, []], 1024, [2048]]
```

Since 256 < 1024 and 256 < 512, insert as the left child of 512.

Takeaways

- Binary trees add one new base case: the empty tree.
- Unpacking with `left, root, right` makes recursive code easier to read.
- Helper functions can be combined to solve new questions.
- Infinity sentinels let empty trees participate safely in comparisons.